Verified Perfect Hashing

A precise existence theorem with explicit resources and probability

Verifiable Algorithms project

8 November 2026

Abstract

There are explicit, fixed word-RAM programs implementing a static dictionary by two-level perfect hashing: a 238-instruction constructor and a 33-instruction query program, both using 64 registers. For an admissible list of N word keys with d distinct values, construction always succeeds, takes at most $10000(N+1)$ expected instructions, and retains at most $64(d+1)$ words. Every query is exact and takes at most 33 instructions. We give only the definitions needed to state the complete implementation guarantee, including finite memory capacity, the probability space, worst-case resources, and the explicit probability bound for deterministic fallback. The existence claim is witnessed by the completed Lean certificate `PerfectHash.certifiedFKS`; preprocessing bounds count executed RAM instructions through successful halt.

1 Model and calling assumptions

Write $[n] = \{0, \dots, n-1\}$. For an input list $x = (x_0, \dots, x_{N-1})$, let $S = \{x_i : i \in [N]\}$ and $d = |S|$. Thus N counts occurrences, $d \le N$ counts distinct keys, and both can be zero. Natural-number resource bounds below are mathematical quantities; they need not themselves fit in one machine word.

Definition 1 (Machine, cost, and randomness)**.** For a width $w \ge 1$, let $U = 2^w$ and $\mathcal{W}_w = [U]$. Keys, addresses, and register values are words in $\mathcal{W}_w$. The machine has 64 registers, mutable ordinary memory $\mu : \mathcal{W}_w \to \mathcal{W}_w$, and separate immutable random memory $\rho : \mathcal{W}_w \to \mathcal{W}_w$. A program is a finite instruction array with a natural-number program counter, initially zero.

The instructions are move, addition, subtraction, multiplication, unsigned remainder, logical right shift, ordinary load/store, random load, conditional branch, jump, and halt. Addition, subtraction, multiplication, and literal conversion wrap modulo U. Remainder by zero returns its dividend; right shift by c is $\lfloor a/2^c \rfloor$, including zero when $c \ge w$. Comparisons are unsigned. Branch and jump targets are literal instruction positions; an out-of-range fetch faults. Each executed instruction, including multiplication, memory operations, and halt, costs one.

For a completed construction let T be its instruction count, L its retained ordinary-memory length, and J its number of random loads. If K distinct random addresses are read, the exposed raw-bit count is $B = wK$. Ordinary space counts distinct ordinary cells plus the 64 registers; program code, the semantic program counter, and execution traces are excluded. Random-memory reservation is accounted for separately.

All probabilities and expectations fix the input, layout, initial registers, and ordinary memory first, and then choose one complete image ρ uniformly from $\Omega_w = \mathcal{W}_w^{\mathcal{W}_w}$. Explicitly, for an event $A \subseteq \Omega_w$ and a real-valued observation g,

$$\mathbb{P}(A) = \frac{|A|}{U^U}, \qquad \mathbb{E}g = \frac{1}{U^U} \sum_{\rho \in \Omega_w} g(\rho).$$

Distinct random addresses therefore contain independent uniform words. Repeated reads of an address return its original value.

Definition 2 (Admissible invocation)**.** A region (b, ℓ) denotes $[b, b+\ell)$ and *fits* when $b+\ell < U$. Two regions (b, ℓ) and (c, k) are *separated* when $b + \ell \le c$ or $c + k \le b$. The strict endpoint condition applies even to an empty region. Put

$$C(N) = 64(N + 1), \qquad R(w, N) = 1 + w\big(2(N + 1) + 1\big).$$

The caller chooses four natural bases $b_{\text{in}}, b_{\text{out}}, b_s, b_\rho$. Its ordinary regions are

$$I = (b_{\text{in}}, N), \quad O = (b_{\text{out}}, C(N)), \quad A = (b_s, C(N)),$$

and its random region is $V = (b_\rho, R(w, N))$. The invocation is *admissible* if I, O, A fit and are pairwise separated, V fits in the separate random address space, and

$$1 \le w, \qquad w < U, \qquad C(N) < U.$$

Initially $\mu(b_{\text{in}} + i) = x_i$ for $i \in [N]$, and registers 0 through 8 contain, in order,

$$(b_{\text{in}}, N, b_{\text{out}}, C(N), b_s, C(N), b_\rho, R(w, N), w).$$

Other registers and ordinary cells are arbitrary. The constructor receives neither d, a distinct input list, nor a successful hash seed.

These are capacity and calling assumptions only. Repeated keys, zero, and $U - 1$ are allowed; scratch memory may be dirty; N^2 need not fit in a word. The assumptions are feasible: the bases $(0, N, N + 64(N + 1), 0)$ work whenever $w \ge 1$ and

$$N + 128(N + 1) < 2^w, \qquad R(w, N) < 2^w.$$

For every list of k-bit keys, zero-extension to any $w \ge \max\{k, 256(N + 1)\}$ suffices. This is a loose sufficient threshold; the two displayed inequalities give the sharper placement test.

Definition 3 (Local perfect-hashing dictionary)**.** A retained descriptor is a pair (b, L) with $L > 0$ and $b + L < U$. A perfect-hashing dictionary for S stores full keys in this region and computes a slot for each member, with different members selecting different slots. Its membership operation returns exactly 1 for $q \in S$ and 0 otherwise, checking occupancy and the stored full key. Nonmembers may collide with members' slots.

The query invocation starts at program counter zero with (b, L, q) in registers 0, 1, and 2; other registers are arbitrary. The answer is returned in register 0. Correctness is *local*: it depends only on the retained cells, so it survives arbitrary overwrites outside $[b, b + L)$. The caller saves the descriptor when making repeated queries.

2 Existence and exact guarantees

The two-level construction groups distinct keys into root buckets and builds a secondary table for each nonempty bucket. The number of root buckets is $m = 2^{\min\{r \in \mathbb{N} : d \le 2^r\}}$, so $m = 1$ for $d = 0$ and $m \le 2(d + 1)$. The implementation tries at most w random seeds for the root and for each child bucket containing at least two distinct keys. After w unsuccessful trials it enumerates all odd word multipliers until one succeeds. Let F be the event that *any* such root or child search enters this deterministic fallback. Empty and singleton children do not search.

Theorem 1 (An explicit, total, uniform implementation)**.** There exist fixed finite programs P, Q, of lengths 238 and 33 respectively, on the machine of Definition 1, implementing a local perfect-hashing dictionary. The arrays and the constants below are chosen before the width, input, layout, initial state, and random image. For every admissible invocation in Definition 2:

(i) **Total construction and correctness.** For every $\rho \in \Omega_w$, P halts successfully and returns (b_{out}, L, d) in registers 0, 1, and 2. The retained region lies within O and is a dictionary for exactly S as in Definition 3. Construction preserves the original input and every ordinary cell outside $O \cup A$.

(ii) **Actual instruction bounds.** The complete run, including duplicate removal, initialization, rejected trials, any enumeration, and its successful halt, satisfies

$$\mathbb{E}T \le 10000(N+1), \qquad \forall \rho, \quad T(\rho) \le 10000\big((N+1)^2 + U(d+1)\big).$$

Every query $q \in \mathcal{W}_w$ takes at most 33 instructions, including halt. It returns exact membership, leaves ordinary memory unchanged, reads only retained cells, and performs no random loads. These claims hold for all queries simultaneously for each completed table, including queries chosen after inspecting that table and after arbitrary changes outside it.

(iii) **Space and access bounds.** For every image, $L \le 64(d+1)$. Output and scratch reservations are each $64(N+1)$ words. At every finite execution prefix, ordinary accesses stay within $I \cup O \cup A$, writes within $O \cup A$, and random accesses within V; no prefix faults. Distinct ordinary addresses accessed plus registers are at most

$$N + 128(N+1) + 64 \le 193(N+1).$$

The retained bound includes headers, directory entries, seeds, occupancy, and full keys. Random reservation is separately $R(w, N)$ words.

(iv) **Randomness resources.** Every random address actually read is fresh, so $K = J$ and $B = wJ$. The completed run satisfies

$$\begin{aligned} J(\rho) &\le 1 + w\big(2(d+1)+1\big), & \mathbb{E}J &\le 1 + 2(d+1), \\ B(\rho) &\le w\big[1 + w(2(d+1)+1)\big], & \mathbb{E}B &\le w\big[1 + 2(d+1)\big]. \end{aligned}$$

(v) **Probability guarantee.** Under the single uniform random-memory law of Definition 1,

$$\boxed{\mathbb{P}[\text{successful construction and correct answers to all queries}] = 1,}$$
$$\mathbb{P}(F) \le \min\left\{1, \frac{m+1}{2^w}\right\} \le \min\left\{1, \frac{2d+3}{2^w}\right\}.$$

The first assertion also holds pointwise for every image. The second is an upper bound on fallback probability, not an equality and not a probability of an incorrect answer. Fallback runs are included in $\mathbb{E}T$.

Thus ordinary construction space is $O(N+1)$, retained space is $O(d+1)$, expected preprocessing time is $O(N+1)$, and worst-case query time is $O(1)$, with uniform constants. The expected-time inequality does not assert that every construction takes at most $10000(N+1)$ instructions. The separate random-memory reservation is $O(w(N+1))$ words, even though only $O(d+1)$ words are read in expectation.

3 Why the concrete witness satisfies the statement

Witness and refinement. Take P to be `PerfectHash.Total.program` and Q to be `PerfectHash.Query.program`. The theorem `PerfectHash.fksCode_correct` proves their complete `DictionarySpec` with constants $(10000, 33, 10000)$ and the packed representation `PerfectHash.Table.representation`. The value `PerfectHash.certifiedFKS` packages these arrays and this proof; `PerfectHash.dictionary_algorithms_exist` derives the existential statement from that value. The arrays are executable data, independent of width and input, and are not extracted from a choice of successful seeds.

The public contract gives construction, time, space, randomness resources, and queries after overwriting unretained memory. The implementation-specific `PerfectHash.Total.fallback_probability` supplies the additional bound on F. The following explains the mathematical ingredients, rather than introducing further prerequisites for the statement.

Perfect placement and storage. Executed deduplication computes the distinct keys before the two-level construction. Hashes multiply by an odd word modulo U, then take a specified number r of high bits. For distinct keys and a uniform odd seed, the proved collision probability is at most $2/2^r$.

The root has m buckets and is accepted when their populations b_j satisfy $\sum_{j<m} b_j^2 \le 6d$. Let q_j be the slot count for bucket j: zero for an empty bucket, one for a singleton, and otherwise the least power of two at least $2b_j^2$. A larger bucket's seed is accepted only if its keys have distinct secondary indices. Consequently $q_j \le 4b_j^2$ and

$$\sum_{j<m} q_j \le 24d, \qquad L = 4 + 4m + 2\sum_{j<m} q_j \le 64(d+1).$$

The four-word header, four words per bucket, and two words per slot account for the entire retained region. Occupancy is stored separately from the key, so all word values are usable as keys.

The packed representation's `Table.Valid.contents` condition says that a selected slot contains a key exactly when that key belongs to the specified bucket and set. If two members selected the same slot, its single stored value would equal both, proving injectivity. Disjoint child arrays give distinct physical slots across buckets. This establishes perfection of the built representation; the fixed query program checks the complete key to reject nonmembers.

Total search and its probability. For both the root and each nontrivial child, the collision estimates give a rejection probability $p \le 1/2$ per independent trial. This also ensures that some odd multiplier succeeds. Sampling maps a uniform raw word to a uniform odd multiplier, with two raw preimages per multiplier. After at most w random trials, exhaustive enumeration visits at most $H = 2^{w-1}$ odd multipliers and therefore succeeds for every image. It keeps its candidate in registers and uses no random loads.

The deduplication seed, the root's w cells, and each child's w cells are disjoint blocks in V. For a fixed stage, the probability that all w samples are rejected is exactly $p^w \le 2^{-w}$. Conditional on the entire root block, the accepted root and all child populations are fixed, even when the root was found by enumeration; each child's block is still independent and uniform. A union bound over the root and at most m children gives $\mathbb{P}(F) \le (m+1)2^{-w}$. This reasoning requires distinct sample addresses, not distinct sampled values. Correctness never relies on avoiding F.

Charged preprocessing. If a stage's per-trial instruction budget is D, its expected random trial count is at most $\sum_{i=0}^{w-1} 2^{-i} \leq 2$; charging all H enumeration candidates on an exhausted image adds at most $H2^{-w}D = D/2$. The formal stage estimate uses the conservative bound $3D$. Accepted root energy makes the sum of the individual child budgets linear in $d+1$. The deduplicator's expected work is linear in the occurrence count $N+1$. The concrete execution proofs also charge rounding, clearing, grouping, metadata, loop control, and halt, yielding the uniform constant 10000. Replacing random and enumeration trial counts by their pointwise maxima gives the displayed all-image time bound.

The Lean evaluator uses sufficient fuel $10000((N+1)^2+U(N+1))$, obtained by substituting N for d in the worst-case bound. This is proof-side fuel, not an input or time charged to the algorithm. Every such run reaches the program's own successful halt, so the expectation counts complete executions rather than truncated prefixes. All claims concern this word-RAM cost model, not the host cost of running the Lean interpreter.

4 Checked sources and reproduction

This statement describes the completed M11 implementation at repository revision `f67e943`, not an open implementation specification. The project pins Lean 4.24.0 and Mathlib revision `f897ebcf72cd16f89ab4577d0c826cd14afaafc7`. Source links target declaration lines in this fixed revision, so later edits do not move their targets. Theorem names in the middle rows are in `PerfectHash.Total`.

Claim	Checked source and entry point
Capacity and calling assumptions	Admissibility.lean: `Admissible`, `canonical_admissible`.
Complete machine contract	Total/Contract.lean: `specification`.
Every-image output and access	Total/Interface.lean: `output`, `prefix_access`.
Expected and worst-case time	Total/Expected.lean: `expected_instructions`; Total/WorstCase.lean: `work_le_worst`.
Expected randomness	Total/ExpectedRandom.lean: `expected_random_reads`, `expected_exposed_bits`.
Fallback probability	Total/FallbackProbability.lean: `fallback_probability`.
Local perfect representation	Representation.lean: `Table.Valid`, `Table.View`, `Table.representation`.
Query correctness	Query/Correctness.lean: `PerfectHash.Query.spec`.
Concrete existential witness	Certified.lean: `PerfectHash.certifiedFKS`, `PerfectHash.dictionary_algorithms_exist`.

From the repository root, run `./check.sh` to build the development, audit theorem and executable dependencies, run boundary and forced-fallback fixtures, and check the exported program arrays. The proof audit permits only Lean's `propext`, `Classical.choice`, and `Quot.sound`; executable definitions are separately audited for no axiom dependencies. The prose explains the checked results and their elementary consequences; it is not an additional Lean proof artifact.

Run `./scripts/build_paper.sh` to regenerate `docs/perfect-hashing-paper.pdf` from this source using Tectonic, `latexmk`, or `pdflatex`. The build fails on unresolved references or overflowing boxes.